سیستم عامل

پروژه شماره 1

مدرس: دکتر کارگهی

طراحان: سجاد تقی زاده,محمدحسین شفیعی,امیرمحمد براری

شبیه‌سازی و ارزیابی سه معماری ساختار سیستم‌عامل

در این پروژه سه روش سازمان‌دهی اجزای سیستم‌عامل را در یک شبیه‌ساز C++17 کنار هم می‌گذاریم. بخش‌های ناقص starter kit را کامل می‌کنید تا یک برنامه کپی فایل، بدون هیچ تغییری، روی هر سه معماری اجرا شود. بعد مسیر فراخوانی‌ها، جابه‌جایی داده و راه گسترش هر معماری را با هم مقایسه می‌کنید.

1. معماری یکپارچه با الهام از لینوکس (Linux-inspired Monolithic)
2. معماری لایه‌ای اکید (Strict Layered Approach)
3. معماری یکپارچه ماژولار (Modular Monolithic)

این سه، تنها معماری‌های سیستم‌عامل نیستند

سیستم‌عامل‌ها به شکل‌های دیگری هم ساخته می‌شوند. ساختار ساده بدون مرز مشخص (مثل MS-DOS)، ریزهسته (microkernel، مثل MINIX 3، Mach و seL4)، هسته ترکیبی (hybrid، مثل Windows NT و XNU در macOS)، اگزوکرنل (exokernel)، یونی‌کرنل (unikernel) و ماشین مجازی و hypervisor نمونه‌هایی از آن‌ها هستند. سیستم‌عامل‌های واقعی هم معمولاً ترکیبی از چند ایده‌اند، نه نمونه خالص یک مدل.

چرا این سه معماری انتخاب شده‌اند؟

- هر سه یک پرسش اصلی را از سه زاویه جواب می‌دهند: اجزای هسته چطور به هم وصل شوند؟ یکپارچه همه اجزا را در یک فضا با فراخوانی مستقیم کنار هم می‌گذارد. لایه‌ای ارتباط را با قاعده‌ای سخت‌گیرانه محدود می‌کند. ماژولار ساختار یکپارچه را نگه می‌دارد ولی اجزا را در زمان اجرا قابل تعویض می‌کند. مقایسه این سه، مصالحه میان سادگی، نظم ساختاری و گسترش‌پذیری را نشان می‌دهد.

- به سیستم‌های واقعی نزدیک‌اند: لینوکس یک هسته یکپارچه با ماژول‌های قابل بارگذاری است، پس دو مدل یکپارچه و ماژولار مستقیماً به سیستمی که هر روز استفاده می‌کنید مربوط‌اند. مدل لایه‌ای کمتر به‌صورت خالص پیاده می‌شود، ولی قاعده لایه‌بندی و information hiding در طراحی بخش‌های هر سیستم‌عاملی (مثل پشته شبکه و درایورها) دیده می‌شود.
- در یک شبیه‌ساز تک‌پردازه‌ای قابل پیاده‌سازی و شمارش‌اند: هر سه را می‌توان با فراخوانی تابع مدل کرد و تفاوتشان را با شمارنده‌های ساده نشان داد.

ریزهسته مهم‌ترین مدلی است که در این پروژه نیامده. در ریزهسته، سرویس‌هایی مثل فایل‌سیستم و درایورها پردازه‌های جدا در فضای کاربرند و با تبادل پیام (IPC) با هم ارتباط دارند. شبیه‌سازی معنادار آن به پردازه، فضای آدرس جدا و ارسال پیام نیاز دارد که در این شبیه‌ساز نیست. به همین دلیل فقط در تمرین تکمیلی و پرسش‌های تحویل به آن پرداخته می‌شود.

هدف پروژه

در درس دیدیم که یک سیستم‌عامل را می‌توان به چند شکل سازمان داد: یکپارچه، لایه‌ای و ماژولار. هدف این پروژه این است که این تفاوت را به‌جای تعریف، در کد ببینید و بسنجید. یک برنامه کاربری ثابت (کپی فایل) را بدون هیچ تغییری روی سه پیاده‌سازی از یک هسته شبیه‌سازی‌شده اجرا می‌کنید و با شمارنده‌های قابل اندازه‌گیری نشان می‌دهید که هر معماری درخواست برنامه را از چه مسیری عبور می‌دهد، داده را کجا جابه‌جا می‌کند، مرز اجزایش کجاست و برای افزودن یا تعویض یک جزء چقدر انعطاف دارد.

کاری که انجام می‌دهید: بخش‌های ناقص starter kit را (که با TODO و YOUR CODE HERE مشخص شده‌اند) در هر سه معماری کامل می‌کنید تا برنامه کپی فایل روی هر سه درست اجرا شود و همه آزمون‌ها PASS شوند. سپس خروجی شمارنده‌های سه معماری را مقایسه می‌کنید و در تحویل حضوری تحلیل خود را توضیح می‌دهید.

دامنه پروژه: قرار نیست یک سیستم‌عامل واقعی بسازید. دیسک، کنسول و موتور ساده فایل‌سیستم از قبل آماده‌اند. مدیریت پردازه و نخ، مدیریت حافظه و حافظه مجازی، همگام‌سازی و ساختارهای دیسکی مثل Inode در این شبیه‌ساز نیستند. گذر به هسته هم با دستور Trap واقعی انجام نمی‌شود، بلکه با فراخوانی تابع مدل می‌شود. همه اعداد مقایسه، شمارش‌های مدل‌شده‌اند و ادعایی درباره سرعت واقعی ندارند.

پس از انجام این پروژه می‌توانید

درباره مفاهیم پایه

- فرق **user space** و **kernel space** و نقش **system call** را توضیح دهید.
- بگویید چرا برنامه به‌جای خود فایل یک **هندل** می‌گیرد و هسته پشت آن چه چیزی نگه می‌دارد.
- سه روش **parameter passing** (رجیستر، بلوک حافظه، پشته) را مقایسه کنید.

درباره هر معماری

- **یکپارچه:** نشان دهید همه درخواست‌ها از یک دروازه (syscall) و یک **dispatch table** می‌گذرند.
- **لایه‌ای:** توضیح دهید چرا هر لایه فقط با لایه زیر خودش حرف می‌زند، و **information hiding** چه کمکی می‌کند.
- **ماژولار:** نشان دهید چطور ماژول‌ها در زمان اجرا ثبت، انتخاب و حذف می‌شوند و چرا حذف ماژولِ دارای فایل باز **ایمن نیست**.

درباره اصول طراحی

- **مکانیزم** را از **پالیسی** جدا کنید.
- بفهمید یک OSApi چطور باعث می‌شود یک برنامه روی هر سه معماری بدون تغییر اجرا شود.
- بدانید **گسترش‌پذیری** یعنی چه و چرا تعویض کنسول یا ماژول بدون تغییر بقیه ممکن است.

درباره مقایسه و ارزیابی

- سه معماری را با **معیارهای قابل شمارش** (layer transition، domain switchها، data copy، عملیات دیسک) مقایسه کنید.
- بدانید چرا این اعداد **مدل‌شده‌اند** و چرا زمان اجرا یا عمق دستی برای رتبه‌بندی کافی نیستند.
- بفهمید چرا پاس شدن تست‌ها به‌تنهایی رعایت معماری را ثابت نمی‌کند و بازبینی کد لازم است.
- معماری‌های دیگر سیستم‌عامل (مثل ریزهسته و هسته ترکیبی) را نام ببرید و پیش‌بینی کنید شمارنده‌های این پروژه در آن‌ها چگونه تغییر می‌کرد.

پرسش‌های تحویل حضوری از همین دستاوردها و از کدی که نوشته‌اید طرح می‌شود.

سناریوی مشترک و رابط برنامه‌نویسی

برنامه copy_file در app/UserPrograms.hpp آماده است. این تابع یک برنامه سطح کاربر است، نه system call، و به این ترتیب کار می‌کند:

1. اگر اندازهٔ قطعه صفر باشد یا مسیر مبدأ و مقصد یکی باشد، فقط یک پیام خطا با console_write می‌نویسد و false برمی‌گرداند.
2. پیام شروع را با console_write در کنسول می‌نویسد.
3. فایل مبدأ را در حالت Read و فایل مقصد را در حالت Write باز می‌کند. اگر باز کردن هر کدام شکست بخورد، پیام خطا می‌نویسد (و در شکست مقصد، handle مبدأ را هم می‌بندد).
4. در یک حلقه می‌خواند و می‌نویسد تا read صفر (EOF) برگرداند. خطای read، خطای write یا نوشتن کمتر از اندازهٔ داده، حلقه را قطع می‌کند.
5. هر دو handle را می‌بندد.
6. در پایان یک پیام با console_write می‌نویسد: در موفقیت تعداد بایت‌های کپی‌شده، و در خطا پیام «Copy aborted with errors».

برنامه فقط از رابط OSApi استفاده می‌کند و هر سه معماری باید همین رابط را پیاده کنند. تغییر برنامه کپی برای جور شدن با یک معماری مجاز نیست.

```
int open(const std::string& path, OpenMode mode);
int read(int handle, std::vector<uint8_t>& out_buffer,
         size_t max_bytes);
int write(int handle, const std::vector<uint8_t>& data);
int close(int handle);
void console_write(const std::string& text);
```

- open در صورت موفقیت یک هندل نامنفی برمی‌گرداند. سرویس فایل استاندارد فقط Read و Write را می‌پذیرد؛ Append و ReadWrite رزرو شده‌اند و با InvalidArgument رد می‌شوند. باز کردن در حالت Write محتوای فایل موجود را پاک می‌کند.
- read و write تعداد بایت‌ها را برمی‌گردانند. در خواندن با طول مثبت، صفر یعنی EOF.
- عملیات صفربایتی روی هندل معتبر و با مجوز درست، صفر برمی‌گرداند و موقعیت فایل را تغییر نمی‌دهد.

- close در موفقیت صفر برمی‌گرداند. در همه توابع، خطا با ثابت منفی متناظر گزارش می‌شود.
- کمبود فضا با NoSpace گزارش می‌شود و نوشتنی که به این دلیل شکست بخورد نباید بلوکی رزرو کند.

ثابت خطا	مقدار	معنا
NotFound	-۱	فایل پیدا نشد
InvalidHandle	-۲	هندل نامعتبر
PermissionDenied	-۳	عملیات با مجوز فعلی جور نیست
NoSpace	-۴	فضای کافی نیست
ServiceUnavailable	-۵	سرویس فعالی در دسترس نیست
Busy	-۶	منبع در حال استفاده است
InvalidArgument	-۷	آرگومان نامعتبر

در کد همیشه از نام ثابت‌ها استفاده کنید، نه از عددشان. unload_module مقدار bool برمی‌گرداند و رد شدن حذف ماژول با false مشخص می‌شود.

starter kit و زیرساخت آماده

همه کار در پوشه starter_kit انجام می‌شود. جاهایی که باید کامل شوند با TODO و YOUR CODE HERE مشخص شده‌اند. قبل از شروع، فایل‌های آماده را بخوانید و قرارداد رابط‌ها را عوض نکنید.

```
starter_kit/
  platform/        OSApi, VirtualHardware, SimpleFileEngine, Tracer
  app/             UserPrograms.hpp
  1_monolithic/    MonolithicKernel, MonolithicOSApi
  2_layered/       Layer0, Layer1, Layer2, Layer3, LayerTypes
  3_modular/       ModuleInterface, ModularKernel, StandardModules, ModularOSApi
  test_runner.cpp
  test_support.hpp
```

Makefile

1. VirtualDisk یک دیسک خام در حافظه با بلوک‌های ثابت ۶۴ بایتی است. فقط شماره بلوک و دو عمل read_block و write_block را می‌شناسد؛ نام فایل در این سطح معنایی ندارد.
2. VirtualConsole متن خروجی را در یک تاریخچه نگه می‌دارد. NullConsoleDevice هر چه بگیرد دور می‌ریزد و برای آزمون گسترش‌پذیری به کار می‌رود.
3. SimpleFileEngine ساخت فایل، پاک کردن محتوای قبلی، تخصیص بلوک و تبدیل نام فایل و موقعیت بایت به بلوک دیسک را انجام می‌دهد. این موتور از طریق callbackهای خواندن و نوشتن بلوک به بقیه سیستم وصل می‌شود. بلوک صفر رزرو است، پس ظرفیت داده برابر است با (تعداد کل بلوک‌ها منهای یک) ضرب در ۶۴ بایت.
4. Tracer مسیر درخواست‌ها، تعداد layer transitionها، رخدادهای data copy، عبور درخواست‌ها از رابط ماژول‌ها و عمق اعلامی فراخوانی را ثبت می‌کند. ثبت درست این رخدادها جزو کار هر معماری است.

این زیرساخت‌ها را از نو بنویسید. رابط OSApi، برنامه کاربر و منطق آزمون‌ها را هم برای پاس کردن مصنوعی تست‌ها تغییر ندهید.

بخش اول: معماری یکپارچه با الهام از لینوکس (20 نمره)

فایل‌های اصلی: monolithic/MonolithicKernel.cpp و monolithic/MonolithicOSApi.cpp

در مدل یکپارچه، سیستم فایل، درایورها و جدول توصیف‌گرهای فایل همه داخل هسته‌اند. پوسته MonolithicOSApi آرگومان‌ها را در SyscallArgs می‌گذارد و درخواست را همراه شماره سرویس به MonolithicKernel::syscall می‌فرستد.

Dispatch table برای syscallها

در setup_syscall_table شماره‌های سرویس را به توابع آماده handle_open، handle_read، handle_write، handle_close و handle_console_write وصل کنید. تابع syscall باید از همین جدول استفاده کند و شماره ناشناخته را با InvalidArgument رد کند. آداپتورها آماده‌اند و لامبدا لازم نیست. مسیری که جدول را دور بزند، حتی اگر خروجی درست بدهد، پذیرفته نیست.

ثبت ورود به هسته و بازگشت به کاربر

در syscall، ورود از UserSpace به MonolithicKernel و بازگشت به کاربر را با record_domain_switch ثبت کنید. بازگشت باید در مسیر خطا هم ثبت شود، از جمله وقتی شماره سرویس نامعتبر است.

record_call_depth(3) فقط یک عمق اعلامی برای توضیح است و نه اندازه‌گیری پشته است و نه ملاک قبولی.

عملیات فایل و جدول توصیف‌گرها

برای هر فایل باز، نام فایل، حالت دسترسی، موقعیت فعلی و باز بودن را در fd_table_ نگه دارید.

1. sys_open: نام خالی و حالت‌های رزرو را با InvalidArgument رد کند. در حالت Read اگر فایل نباشد NotFound بدهد. در حالت Write فایل موجود را خالی کند یا فایل تازه بسازد. هندل جدید با موقعیت صفر شروع می‌شود.
2. sys_read: هندل را بررسی کند و خواندن از هندلی را که در حالت Write باز شده رد کند. بعد از آن، طول صفر را با صفر جواب دهد و برای طول مثبت بافر تهی را رد کند. بعد از خواندن موفق، موقعیت فایل را جلو ببرد.
3. sys_write: مثل خواندن، هندل و مجوز و بافر را بررسی کند، نوشتن روی هندلی را که در حالت Read باز شده رد کند و بعد از نوشتن موفق موقعیت را جلو ببرد.
4. sys_close: هندل را بررسی کند و آن را بسته علامت بزند. بستن هندل نامعتبر یا هندلی که قبلاً بسته شده باید InvalidHandle بدهد.

هر خواندن یا نوشتن موفق را با record_data_copy و جهت درست (KernelBuffer به UserBuffer برای خواندن و برعکس برای نوشتن) ثبت کنید.

پوسته رابط کاربر

متدهای MonolithicOSApi را با پر کردن SyscallArgs و صدا زدن kernel_.syscall کامل کنید. buffer مقصد خواندن است و write_buffer ورودی فقط‌خواندنیِ نوشتن؛ داده write را مستقیم در write_buffer بگذارید و به const_cast نیازی نیست. در خواندن، بافر خروجی را به اندازه نتیجه resize کنید و اگر نتیجه صفر یا خطا بود خالی‌اش کنید.

بخش دوم: معماری لایه‌ای (20 نمره)

در این معماری هر لایه فقط از رابط لایه درست زیر خودش استفاده می‌کند. اگر لایه فایل‌سیستم مستقیم به

سخت‌افزار دسترسی داشته باشد، معماری نقض شده، حتی اگر خروجی برنامه درست باشد.

لایه	مولفه	مسئولیت
۳	Layer3_OSApi	ساخت بلوک پارامتر، فرستادن درخواست به لایه ۲ و تحویل نتیجه به برنامه
۲	Layer2_FileSystem	مدیریت فایل و هندل با کمک درایور لایه ۱
۱	Layer1_Drivers	خواندن و نوشتن سکتور و فرستادن خروجی نمایشگر به لایه ۰
۰	Layer0_Hardware	دسترسی خام به دیسک و کنسول مجازی

ارتباط فقط میان لایه‌های مجاور

در Layer1_Drivers.cpp توابع driver_read_sector و driver_write_sector را به hw_.raw_read_block و hw_.raw_write_block وصل کنید. خروجی نمایشگر هم از driver_write_display به hw_.raw_write_console می‌رسد. لایه ۲ فقط از drivers_ استفاده می‌کند و لایه ۳ درخواست را فقط به لایه ۲ می‌دهد.

information hiding

درایور فقط شماره سکتور و بافر خام را می‌شناسد. نام فایل، مجوز دسترسی و موقعیت فایل کار لایه فایل‌سیستم است. اعضای کلاس‌ها و وابستگی‌های میان لایه‌ها هم باید همین مرز را نگه دارند.

بلوک پارامتر

در Layer3_OSApi.cpp برای هر سرویس یک ParameterBlock بسازید:

- op: نوع عملیات
- path و text: رشته‌های std::string
- buffer: مقصد خواندن؛ write_buffer: ورودی نوشتن؛ length: طول داده
- mode و handle: مطابق درخواست

بلوک را به layer2_.process_request pb بدهید و نتیجه را از pb.status بخوانید. در read، چون لایه ۲ فقط یک اشاره‌گر خام به بافر دارد، تنظیم اندازه بافر خروجی با لایه ۳ است: قبل از فرستادن درخواست، بافر را به اندازه

max_bytes آماده کنید و بعد از برگشتن، آن را به اندازه نتیجه واقعی resize کنید یا اگر نتیجه صفر یا خطا بود خالی‌اش کنید.

در Layer2_FileSystem.cpp تابع process_request را کامل کنید تا بر اساس pb.op یکی از توابع handle_close، handle_write، handle_read، handle_open یا handle_console_write را صدا بزند و نتیجه را در pb.status بگذارد. برای عملیات ناشناخته InvalidArgument برگردانید.

ثبت layer transitionها و data copy

- هر فراخوانی رو به پایین میان دو لایه مجاور با record_layer_transition ثبت می‌شود.
- layer transition ۳ به ۲ را خودتان در هر متد Layer3_OSApi ثبت کنید.
- layer transition ۲ به ۱ (برای سکتورها و نمایشگر) و ورود و بازگشت user/kernel در لایه ۲ از قبل نوشته شده؛ آن را نگه دارید.
- layer transition ۱ به ۰ را خودتان در لایه ۱ ثبت کنید، هم برای خواندن و نوشتن سکتور و هم برای driver_write_display.
- درایور فقط وقتی عملیات بلوک موفق بود، انتقال مدل‌شده ۶۴ بایتی را با record_data_copy ثبت کند.

عمق اعلامی اینجا هم فقط برای توضیح است. شمارنده layer transitionها از شمارنده domain switch جداست.

بخش سوم: معماری یکپارچه ماژولار (20 نمره)

فایل‌های اصلی: modular/ModularKernel.cpp و modular/StandardModules.cpp

هسته ماژول‌ها را ثبت و انتخاب می‌کند و درخواست‌ها را به آن‌ها می‌رساند. پیاده‌سازی سرویس‌ها و پالیسی دسترسی داخل ماژول‌هاست. IKernelModule نام، نوع و متدهای on_load و on_unload را تعریف می‌کند. IFileSystemModule عملیات فایل و get_active_handle_count را دارد و IConsoleModule خروجی کنسول را.

مکانیزم، پالیسی و مسیر سرویس‌ها

سرویس‌های sys_write، sys_read، sys_open و sys_close به active_fs_ می‌روند و خروجی کنسول به active_console_. اگر ماژول فایل‌سیستم فعالی نباشد، عملیات فایل ServiceUnavailable برمی‌گرداند. هسته نباید برای تصمیم درباره مجاز بودن نوشتن به نام یا نوع یک ماژول خاص وابسته باشد. این قسمت در starter kit آماده است؛ آن را بخوانید و هنگام تکمیل بقیه بخش‌ها قراردادش را حفظ کنید.

هر درخواستی که به یک ماژول فعال می‌رسد، با record_module_dispatch ثبت می‌شود و شمارنده module_dispatches را زیاد می‌کند. این ثبت در کد آماده هسته انجام شده و لازم نیست آن را تغییر دهید. در معماری‌های یکپارچه و لایه‌ای هیچ درخواستی از رابط ماژول عبور نمی‌کند و این شمارنده صفر می‌ماند. پس این شمارنده تفاوت قابل اندازه‌گیری میان معماری یکپارچه و ماژولار را نشان می‌دهد، بی‌آنکه زمان یا سرعت را ادعا کند.

ثبت و انتخاب ماژول

ثبت و انتخاب ماژول هم آماده است. نام تکراری، ماژول تهی، نام ناشناخته و انتخاب ماژولی با نوع نادرست رد می‌شوند. تا وقتی ماژول فایل‌سیستم فعال هندل باز دارد، نمی‌شود آن را عوض کرد. تعویض کنسول به NullConsoleModule در زمان اجرا بدون تغییر منطق هسته انجام می‌شود.

حذف امن ماژول

تابع unload_module را کامل کنید:

- اگر ماژولی با این نام ثبت نشده، false برگردانید.
- اگر ماژول از نوع فایل‌سیستم است و 0 < ()get_active_handle_count، حذف را رد کنید، رخداد UNLOAD_REJECTED را ثبت کنید و false برگردانید. ماژول باید همچنان قابل استفاده بماند.
- در غیر این صورت on_unload را صدا بزنید، اگر این ماژول سرویس فعال بوده اشاره‌گر فعال را پاک کنید، ماژول را از جدول ثبت حذف کنید، رخداد UNLOAD_SUCCESS را ثبت کنید و true برگردانید.

رفتار هسته هنگام خرابی ماژول

آزمون‌ها یک ماژول فایل‌سیستم معیوب (FaultyFSModule) می‌سازند که همه عملیاتش خطا برمی‌گرداند. با کد کامل‌شده باید این رفتارها برقرار باشد:

- هسته کد خطای ماژول را بدون تغییر به برنامه می‌رساند.
- ورود و بازگشت user/kernel در این فراخوانی‌های ناموفق هم متوازن ثبت می‌شود و هیچ data copy مدل‌شده‌ای ثبت نمی‌شود.
- بقیه سرویس‌ها، مثل کنسول، کار می‌کنند.
- با تعویض به یک ماژول سالم و حذف ماژول معیوب، سیستم بازیابی می‌شود.
- ماژولی که هنوز هندل باز گزارش می‌کند، حتی اگر معیوب باشد، به‌زور حذف یا جایگزین نمی‌شود.

این رفتارها به unload_module شما وابسته‌اند. درباره پیامد آخرین مورد فکر کنید: ایمنی هسته در برابر حذف

ماژول با هندل باز، در عمل می‌تواند باعث شود یک ماژول خراب برای همیشه در سیستم بماند. این بحث جزو پرسش‌های تحویل است و آزمون سقوط واقعی هسته نداریم.

پالیسی فایل‌سیستم فقط‌خواندنی

در ReadOnlyFSModule::open_file، مثل ماژول استاندارد، اول مسیر خالی (یا ماژول بارگذاری‌نشده) را با InvalidArgument رد کنید. بعد هر حالتی غیر از OpenMode::Read، از جمله Append و ReadWrite، با PermissionDenied رد شود. سپس اگر فایل نبود NotFound بدهید و در غیر این صورت هندل معتبر بسازید و شمارنده هندل‌های فعال را زیاد کنید. write_file باید همه درخواست‌های نوشتن را با PermissionDenied رد کند. این محدودیت داخل خود ماژول اعمال می‌شود، نه در هسته.

در آزمون پالیسی، ماژول فقط‌خواندنی همان موتور فایل StandardFSModule را به اشتراک می‌گیرد تا بتواند فایلی را که قبلاً ساخته شده بخواند. هدف این است که سیاست دسترسی عوض شود بی‌آنکه مکانیزم هسته تغییر کند.

بخش نهایی: ردیابی و مقایسه معماری‌ها

آزمون مقایسه روی هر سه معماری یک فایل ۶۵ بایتی را با قطعه‌های ۶۴ بایتی کپی می‌کند. ساختن فایل مبدا و بررسی محتوای مقصد در شمارش حساب نمی‌شوند. خروجی شمارنده‌ها را برای تحویل حضوری تحلیل کنید. زمان اجرا به سیستم و کامپایلر بستگی دارد و در نمره اثری ندارد.

معیار	معنا در این شبیه‌ساز
domain_switches	تعداد ورود و بازگشت user/kernel؛ دو رخداد برای هر فراخوانی API
data_copies	تعداد رخدادهای data copy مدل‌شده؛ عملیات صفربایتی یا ناموفق شمرده نمی‌شود
layer_transitions	تعداد فراخوانی‌های ثبت‌شده رو به پایین میان لایه‌های مجاور
module_dispatches	تعداد درخواست‌هایی که از رابط یک ماژول عبور کرده‌اند؛ فقط در معماری ماژولار غیر صفر است
data_bytes	جمع اندازه رخدادهای data copy مدل‌شده، بر حسب بایت

معیار	معنا در این شبیه‌ساز
disk read_ops / write_ops	تعداد خواندن و نوشتن موفق بلوک در دیسک مجازی

در آزمون مقایسه، معماری ماژولار ۱۱ فراخوانی API دارد و module_dispatches آن هم ۱۱ است؛ در دو معماری دیگر صفر است. data_copies و data_bytes رخدادهای مدل‌شده‌اند و تعداد واقعی memcpy یا بایت‌های کپی‌شده در حافظه را نشان نمی‌دهند. module_dispatches هم یک شمارنده مدل‌شده است. عمق فراخوانی دستی اعلام می‌شود و فقط برای توضیح است؛ قبولی یا سریع‌تر بودن بر اساس آن تعیین نمی‌شود. پیامد خرابی هسته فقط در بحث مفهومی مطرح است. گسترش‌پذیری با تعویض واقعی دستگاه و ماژول سنجیده می‌شود.

قرارداد domain switch در هر سه معماری یکی است: هر فراخوانی API یک ورود و یک بازگشت دارد، حتی وقتی خطا می‌دهد. فراخوانی میان لایه‌ها جدا شمرده می‌شود. read_ops و write_ops را خود دیسک مجازی می‌شمارد و به ثبت‌های Tracer ربطی ندارند.

آزمون‌ها توازن ورود و بازگشت، مجاز بودن ترتیب layer transitionها، اندازه رخدادهای انتقال و عملیات دیسک را با همین قرارداد مقایسه می‌کنند. پاس شدن تست‌های رفتاری به‌تنهایی نشان نمی‌دهد معماری رعایت شده؛ وابستگی هدرها، اعضای کلاس‌ها و مسیر واقعی فراخوانی‌ها در بازبینی کد هم بررسی می‌شوند. هر رخداد ثبت‌شده باید با یک عملیات واقعی در کد متناظر باشد.

آزمون‌ها و روش اجرا (25 نمره)

test_runner.cpp همراه با test_support.hpp پنج گروه آزمون دارد. تعداد ارزیابی‌ها و نتیجه نهایی هنگام اجرا چاپ می‌شود و همه باید PASS شوند. چند نکته:

- بعضی آزمون‌ها اگر مرحله آماده‌سازی‌شان شکست بخورد ادامه پیدا نمی‌کنند، پس تا کد کامل نشده، تعداد کل ارزیابی‌های چاپ‌شده کمتر از عدد نهایی است.
- بعضی آزمون‌ها زیرساخت آماده را می‌سنجند یا با starter kit هم PASS می‌شوند. PASS شدن یک آزمون به این معنا نیست که بخش مربوط کامل شده؛ همه TODOها باید پیاده شوند.

برای ساخت و اجرا :

```
cd starter_kit
make run
```

گروه	موضوع
۱	کپی فایل خالی و دودویی با اندازه‌های ۰، ۱، ۶۳، ۶۴، ۶۵، ۱۲۷، ۱۲۸، ۱۲۹ و ۳۲۷ بایت و قطعه‌های ۱، ۱۷، ۶۴ و ۱۰۰ بایتی
۲	قرارداد رخدادها، توازن layer transition، domain switch میان لایه‌های مجاور، عبور از رابط ماژول و شمارش عملیات دیسک
۳	خطاها، مجوزها، EOF، عملیات صفربایتی، موقعیت مستقل هندل‌ها، کمبود فضا و آزاد شدن بلوک‌ها
۴	تعویض دستگاه کنسول در مدل لایه‌ای و تعویض ماژول کنسول در زمان اجرا
۵	ثبت و انتخاب درست ماژول، رد حذف و تعویض هنگام استفاده، پاک‌سازی بعد از حذف، سیاست فقط‌خواندنی و رفتار هسته هنگام خرابی ماژول

تمرین تکمیلی: معماری‌های دیگر سیستم‌عامل (15 نمره)

یک فایل کوتاه با نام OTHER_ARCHITECTURES.md در ریشه starter_kit بنویسید (حداکثر حدود یک صفحه) و در آن:

1. دست‌کم سه معماری سیستم‌عامل غیر از سه مدل این پروژه را نام ببرید (مثلاً ریزهسته، هسته ترکیبی، اگزوکرنل، یونی‌کرنل، ساختار ساده، ماشین مجازی). برای هر کدام یک سیستم‌عامل واقعی یا پژوهشی مثال بزنید و در دو سه جمله ایده اصلی‌اش را بگویید.
2. برای هر کدام یک مزیت و یک عیب اصلی نسبت به معماری یکپارچه بنویسید.
3. فرض کنید برنامه copy_file روی یک ریزهسته اجرا شود که فایل‌سیستم و درایور دیسک در آن دو سرور جدا در فضای کاربرند. مسیر یک فراخوانی read را قدم‌به‌قدم بنویسید و پیش‌بینی کنید domain_switches و data_copies نسبت به معماری یکپارچه بیشتر می‌شوند یا کمتر، و چرا.
4. در یکی دو جمله بگویید لینوکس، Windows و macOS را در کدام دسته قرار می‌دهید و چرا هیچ‌کدام نمونه خالص یک مدل نیست.

این فایل کد ندارد و آزمون خودکار ندارد. در تحویل حضوری درباره آن سؤال می‌شود و پاسخ باید با بیان خودتان باشد.

موضوعات پرسش‌های تحویل حضوری

در تحویل حضوری کد شما اجرا و بازبینی می‌شود و از دستاوردهای فهرست‌شده در ابتدای متن پرسش می‌شود، از جمله:

- تفاوت user space و kernel space و نقش جدول syscall
- دلیل ارتباط لایه‌ها فقط با لایه مجاور و مفهوم information hiding
- نحوه کار ParameterBlock و مقایسه روش‌های parameter passing
- دلیل ناایمن بودن حذف ماژولی که هندل باز دارد و پیامد این ایمنی
- تفاوت مکانیزم و پالیسی
- تحلیل شمارنده‌های آزمون مقایسه و اینکه چرا مدل‌شده‌اند و زمان اجرا ملاک نیست
- معماری‌های دیگر سیستم‌عامل و پاسخ‌های تمرین تکمیلی
- و دیگر موضوعات مرتبط با پروژه

ممکن است درباره تک‌تک خطوط کد و تصمیم‌های طراحی سؤال شود . سوالات هر بخش نیز تاثیر مستقیم روی نمرات همان بخش دارد.

نکات و نحوه تحویل

1. کد باید C++17 باشد و با g++ و Makefile در Linux ساخته و اجرا شود. فایل‌های ساخت را همراه پروژه نگه دارید.
2. کار را در یک مخزن خصوصی GitHub با کامیت‌های منظم و معنادار پیش ببرید. پوشه .git را هنگام آماده کردن فایل تحویل پاک نکنید.
3. فایل فشرده فقط شامل محتوای تکمیل‌شده starter_kit و فایل‌های لازم برای ساخت و اجرای آزمون‌ها باشد. خروجی برنامه و جواب پرسش‌های تحویل حضوری در جلسه تحویل بررسی می‌شود و گزارش جداگانه لازم نیست.
4. نام فایل تحویل را به این شکل انتخاب کنید و آن را در صفحه درس بارگذاری کنید:

   `OS-Project1-<StudentNumber>.zip`
5. هر فرد باید به همه بخش‌های کد و مفاهیم مسلط باشد. در تحویل حضوری ممکن است درباره تک‌تک

خطوط کد و تصمیم‌های طراحی سوال شود. بخش کدنویسی کوتاه است و بخش مهم نمره، پرسش‌های شفاهی و بازبینی کد است.

6. سوال‌هایتان را در گروه درس یا مستقیم از طراحان بپرسید. توضیحاتی که در این کانال‌ها اعلام شود جزو صورت پروژه است.
7. کدها از نظر شباهت بررسی می‌شوند و با موارد تقلب طبق مقررات درس و دانشگاه برخورد می‌شود.

موفق باشید.